Inferix: A FLOP-Based Token Framework for Decentralized AI Access

Aslan Kose
Independent Researcher
github.com/aslankose

March 2026

Author Note. The author is an IT professional with experience in the pharmaceutical industry, where the intersection of artificial intelligence, regulatory responsibility, and resource ethics is encountered firsthand. This work was motivated by a broader concern about the sustainability of AI infrastructure at a civilizational scale. Modern AI systems impose significant and growing demands on electrical grids and water supplies — resources that are neither infinite nor equitably distributed. The concentration of these demands in large centralized facilities exacerbates existing inequalities in both energy access and AI access. The Inferix framework emerged from the conviction that distributing computational resources across voluntary participants is not merely a technical optimization but a meaningful step toward a more equitable and sustainable AI ecosystem — one in which the communities that bear the costs of AI infrastructure are also empowered to benefit from it.

Abstract

The rapid advancement of artificial intelligence has created a significant disparity in computational access, where high-quality AI inference and training resources remain concentrated among well-funded institutions and technology corporations. Existing decentralized compute networks, while technically promising, are predominantly designed around cryptocurrency incentive structures that prioritize financial speculation over equitable resource distribution.

This paper proposes **Inferix**, a decentralized AI compute-sharing framework in which participants contribute idle processing power in exchange for access tokens denominated in floating-point operations (FLOPs). Each contributing node is benchmarked at registration and assigned a hardware efficiency multiplier, ensuring that token issuance reflects normalized computational utility rather than raw clock metrics. Earned tokens

may subsequently be redeemed for AI inference through a **hybrid inference architecture** that combines a custom distributed layer-sharding pipeline across volunteer nodes with a centralized fallback layer, routing requests through the volunteer network when sufficient capacity is available and gracefully degrading to centralized inference otherwise.

A key property of the proposed architecture is its inherent temporal and geographic distribution. Because contributor nodes span multiple time zones, computational load naturally migrates toward regions where electricity demand and cost are lowest at any given hour, aligning AI workloads with periods of grid surplus. This produces a passive energy optimization effect without requiring explicit scheduling infrastructure.

We formalize the token issuance and redemption model, define the hardware classification methodology, discuss verification and anti-gaming mechanisms, and analyze the sustainability of the proposed ecosystem. The framework is designed to be open-source, hardware-agnostic, and deployable without blockchain infrastructure, making it accessible to researchers, educational institutions, and underserved communities globally.

Keywords: decentralized computing, AI infrastructure, Inferix, FLOP-based tokens, sustainable AI, volunteer computing, energy-aware scheduling, distributed inference

1 Introduction

Artificial intelligence has transitioned from an academic curiosity to a foundational infrastructure layer underpinning scientific research, industrial automation, healthcare, and education. However, the computational resources required to train and serve modern AI models have grown at a pace that far outstrips the capacity of most research institutions, universities, and independent developers. As of 2024, training a frontier large language model requires tens of thousands of specialized accelerators running continuously for months, with associated costs in the tens to hundreds of millions of dollars [2]. Even inference — the comparatively modest task of querying a trained model — demands GPU clusters that remain economically inaccessible to a significant portion of the global research community.

This centralization of AI compute has profound consequences. It concentrates the ability to develop and deploy AI among a small number of well-capitalized actors, creates single points of failure and control in critical AI infrastructure, and systematically excludes researchers and institutions in developing economies from participating meaningfully in the AI revolution. The result is not merely an economic inefficiency but an epistemic one: the diversity of problems that AI is applied to is constrained by the diversity of those who can afford to

apply it.

Decentralized compute networks have emerged as a potential remedy. Projects such as Bittensor [3], Gensyn [4], and Render Network [5] demonstrate that idle GPU resources distributed across voluntary participants can be aggregated into meaningful computational pools. However, these systems are uniformly built on cryptocurrency incentive models, where contributors are rewarded with tradeable tokens whose value is determined by speculative markets. This introduces volatility, regulatory complexity, and a misalignment of incentives — participants are motivated by financial return rather than by a desire to expand AI access.

We propose a fundamentally different incentive structure. In the **Inferix** framework, contributed compute is rewarded not with financially tradeable tokens but with access credits — redeemable exclusively for AI inference on the network. The unit of account is the floating-point operation (FLOP), providing an objective, hardware-agnostic measure of computational contribution and consumption. A contributor who donates one teraFLOP-hour of idle compute earns precisely enough credits to consume one teraFLOP-hour of AI inference. The system is a compute cooperative, not a compute marketplace.

Beyond access equity, this architecture exhibits a valuable emergent property: geographic and temporal load distribution. Because contributor nodes are globally distributed across time zones, the aggregate compute supply available to the network at any given moment draws from regions where it is currently off-peak — where electricity is cheapest, grid demand is lowest, and renewable generation may be at its highest. This passive alignment of AI workloads with low-demand electricity periods represents a meaningful contribution to sustainable computing, achieved without explicit energy-aware scheduling.

The remainder of this paper is organized as follows. Section 2 reviews related work in decentralized compute, federated learning, and sustainable AI infrastructure. Section 3 describes the system architecture. Section 4 formalizes the FLOP-based token model. Section 5 defines the hardware contribution multiplier methodology. Section 6 addresses security and fairness. Section 7 analyzes energy and grid implications. Section 8 discusses ecosystem sustainability and monetization. Section 10 concludes with directions for future work.

2 Related Work

The problem of distributing computational workloads across voluntary participants has a long history, and several distinct lines of research inform the design of the Inferix framework. We organize the related work into four areas: volunteer computing, federated learning, decentralized compute networks, and sustainable AI infrastructure.

2.1 Volunteer Computing

The earliest systematic effort to harness idle consumer hardware for scientific computation was BOINC (Berkeley Open Infrastructure for Network Computing) [7], which has powered projects such as SETI@home, Folding@home, and Rosetta@home since the late 1990s. BOINC demonstrated that millions of volunteer nodes could collectively deliver petaFLOP-scale computation for scientifically meaningful workloads. However, BOINC operates on a purely altruistic model — contributors receive no formal compensation or access rights in return. This limits sustained participation and creates an inherent asymmetry between contributors and beneficiaries. The Inferix framework extends the BOINC philosophy by introducing a reciprocal access model, converting altruistic contribution into a durable, spendable asset.

2.2 Federated Learning

Federated learning [8] distributes model training across participant devices without centralizing raw data, addressing privacy concerns in sensitive domains such as healthcare and personal communication. Frameworks such as TensorFlow Federated [9] and PySyft [10] have demonstrated practical deployments at scale. While federated learning shares our interest in distributing AI computation, it is architecturally focused on training rather than inference, and it does not address the question of access equity — participants contribute compute but do not earn the right to use the resulting model. Furthermore, federated learning requires tight coordination of gradient updates, making it sensitive to node heterogeneity and dropout, challenges that the Inferix framework avoids by decoupling contribution from task execution.

2.3 Decentralized Compute Networks

Several projects have approached decentralized AI compute through blockchain-based incentive structures. Bittensor [3] introduces a peer-to-peer network in which nodes serve AI model outputs and are rewarded with TAO tokens based on the assessed quality of their responses, as judged by validator nodes. While innovative, Bittensor's incentive model is subject to speculative token valuation and introduces complexity around validator collusion and quality assessment subjectivity. Gensyn [4] focuses specifically on decentralized machine learning training, using probabilistic verification to confirm that nodes have honestly executed assigned training tasks, rewarding contributors with tradeable tokens. Render Network [5] and Akash Network [6] provide decentralized GPU marketplaces where contributors lease idle hardware for cash or token compensation. These platforms demonstrate the technical feasibility of aggregating heterogeneous consumer hardware for AI workloads, but they remain fundamentally marketplace models driven by financial incentives rather than access equity. The Inferix framework draws on the architectural insights of these projects while replacing

speculative token economies with a closed-loop FLOP credit system.

2.4 Distributed Inference

The most technically proximate prior work to Inferix is Petals [1], a framework for collaborative inference over large language models by distributing transformer layers across volunteer machines. Each Petals node loads a contiguous slice of model layers and passes intermediate activations to the next node in the pipeline, enabling inference over models that would not fit on any single volunteer device. Petals demonstrates that distributed volunteer inference is technically feasible and that heterogeneous consumer hardware can collectively serve state-of-the-art models at practical latencies.

However, Petals operates on a purely altruistic participation model. Contributors receive no compensation or access rights in return for hosting model layers, creating the same sustainability problem observed in BOINC: participation depends entirely on goodwill and may decline as novelty fades. Furthermore, Petals does not address the question of equitable access — all users access the network equally regardless of whether they contribute any compute themselves, providing no incentive for resource-constrained institutions to participate as contributors rather than purely as consumers.

Inferix addresses both limitations through a custom distributed layer-sharding architecture with an integrated FLOP-based token economy. Contributors earn access credits proportional to the verified inference FLOPs they serve, and spend those credits to access inference on the same network. The distinction is architectural: Petals solves the technical problem of distributed inference, while Inferix solves the economic and governance problem of making distributed inference sustainable and equitably accessible.

2.5 Sustainable AI Infrastructure

The energy consumption of AI systems has attracted increasing scrutiny. Training a single large language model has been estimated to produce carbon emissions comparable to the lifetime emissions of several automobiles [11]. Patterson et al. [12] identify geographic placement of data centers and the carbon intensity of local electricity grids as primary levers for reducing AI's environmental footprint. Dodge et al. [13] further demonstrate that the timing of computation relative to renewable energy availability significantly affects net emissions. These findings motivate the energy-aware properties of the Inferix architecture. Unlike centralized data centers fixed to a single geographic location and grid, a globally distributed volunteer network inherently shifts active computation toward nodes operating in off-peak electricity periods, providing passive alignment with low-carbon grid conditions without requiring explicit carbon-aware scheduling infrastructure.

3 System Architecture

The Inferix framework consists of four principal components: the contributor client, the coordination layer, the token ledger, and the inference cluster. Figure 1 illustrates the high-level architecture and the interactions between these components. We describe each component in turn.

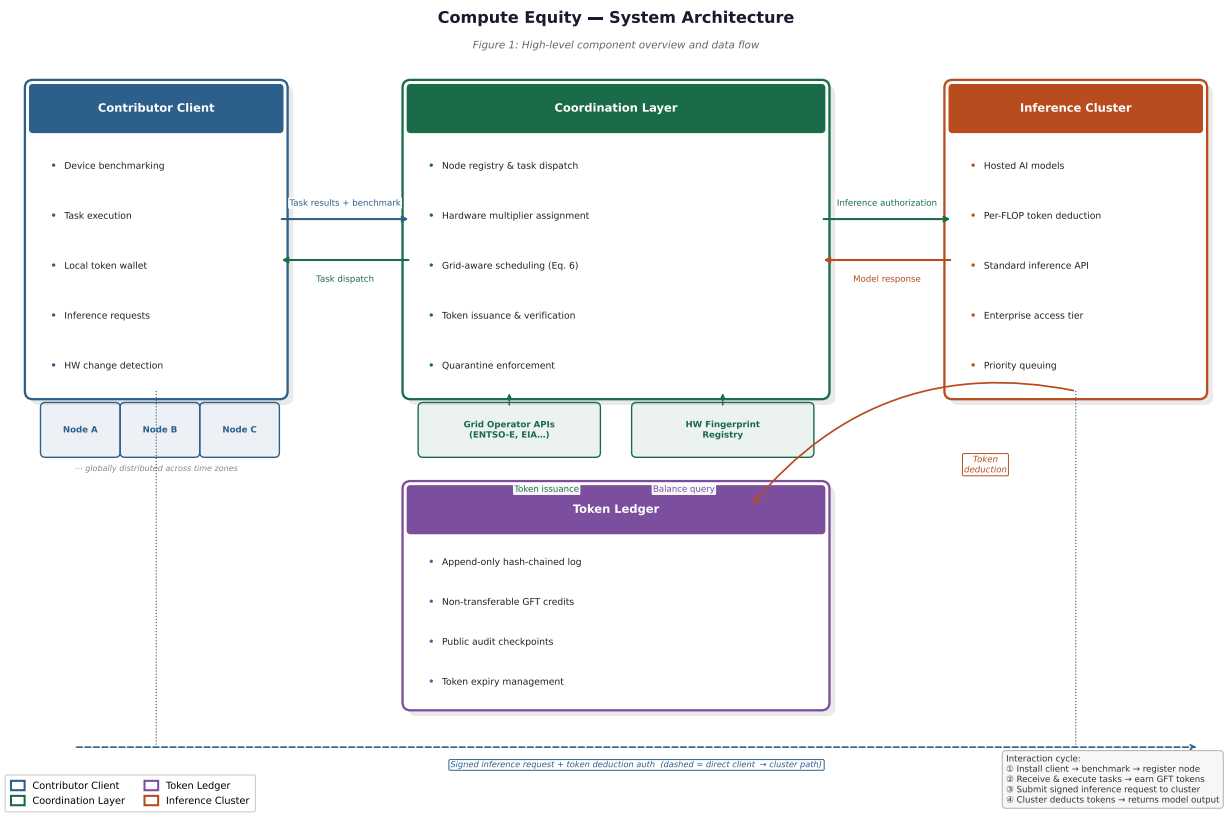

Figure 1: High-level system architecture of the Inferix framework, showing the four principal components and their interactions. Solid arrows represent primary data flows; the dashed arc represents the direct signed inference request path from contributor client to inference cluster.

3.1 Contributor Client

The contributor client is a cross-platform daemon application distributed as a single Python script, installable on Windows, macOS, and Linux with no dependencies beyond the standard Python runtime and the `requests` library. It is responsible for five functions: device benchmarking, shard registration, inference layer serving, reliability challenge response, and

token accounting.

Upon installation, the client executes a standardized benchmarking routine measuring matrix multiplication throughput, memory bandwidth, and inference latency. The composite AI Performance Index score is submitted to the coordination layer, which assigns a hardware efficiency multiplier as described in Section 5. The client also detects GPU type — distinguishing discrete, integrated, and Apple Silicon architectures — and applies a hardware penalty to benchmark scores for integrated GPU and CPU-only configurations, preventing overestimation of AI compute capability on non-specialist hardware.

The client's primary contribution mechanism is *inference layer serving*: upon registration, each node loads a contiguous slice of a target model's transformer layers into local memory and exposes an activation server on a configurable port. The coordination layer's shard scheduler maintains a registry of active nodes and their layer assignments, identifying complete pipelines — sets of nodes whose layer ranges collectively cover all transformer layers with no gaps. When a complete pipeline exists, inference requests are routed through it rather than to the centralized fallback.

A secondary contribution mechanism is *reliability challenge response*: the coordination layer periodically dispatches inference challenge tasks — known prompts with pre-computed expected activation hashes — to contributor nodes. Nodes that return incorrect activation hashes have their reliability factor reduced according to the formula defined in Section 4. This mechanism detects nodes returning fabricated or incorrect activations without performing the assigned computation.

Hardware profile changes trigger a quarantine period during which token issuance is suspended. The quarantine duration scales logarithmically with the magnitude of the hardware upgrade, as defined in Section 6, preventing hardware swap attacks. The client monitors the shard registration heartbeat and automatically re-registers with the coordination layer following a restart, ensuring continuity of service without manual intervention.

3.2 Coordination Layer

The coordination layer is the central orchestration service of the framework. It maintains the registry of active contributor nodes, dispatches compute tasks, verifies task completion, issues tokens, and manages the global token ledger. It is the only centralized component in the architecture, and its design prioritizes auditability and fault tolerance over full decentralization.

Task dispatch follows a scheduling policy that incorporates three factors: node availability,

node efficiency class, and geographic electricity demand. The coordination layer maintains a continuously updated map of electricity grid load by region, derived from publicly available grid operator APIs. When dispatching tasks, the scheduler preferentially assigns workloads to nodes located in regions currently experiencing low grid demand, as these nodes are most likely to be operating on surplus or renewable generation. This scheduling policy is described in detail in Section 7.

The coordination layer exposes a public API through which contributor clients register, receive tasks, submit results, and query token balances. All communications between the client and coordination layer are authenticated using asymmetric key pairs generated during client installation, preventing impersonation and token forgery.

3.3 Token Ledger

The token ledger is an append-only log of all token issuance and redemption events maintained by the coordination layer. Each entry records the contributor identifier, the number of FLOPs delivered, the applicable hardware multiplier, the tokens issued, and a cryptographic hash of the preceding entry, forming a tamper-evident chain. The ledger is periodically checkpointed and published to a public repository, allowing any participant to independently audit the total token supply and verify that issuance has followed the defined formula.

Tokens are non-transferable and non-tradeable by design. A token balance is associated exclusively with the contributor wallet that earned it and cannot be assigned, sold, or exchanged with other participants. This constraint is enforced at the ledger level and is fundamental to the access-equity objective of the framework.

3.4 Inference Cluster

The inference cluster is the component responsible for serving AI model responses to token-holding contributors. The framework implements a **hybrid inference architecture** that combines two complementary backends: a custom distributed layer-sharding pipeline across volunteer nodes, and a centrally operated fallback layer backed by commercial inference APIs.

3.4.1 Distributed Layer-Sharding Pipeline

The primary inference backend distributes transformer model layers across volunteer contributor nodes. Each participating node loads and serves a contiguous slice of the model's transformer layers — determined at registration time based on available VRAM and the current coverage gaps in the shard registry. During inference, input tokens are passed to the first node in the pipeline, which processes its assigned layers and passes intermediate activations to the next node. This continues until the final node generates output tokens.

The coordination layer maintains a shard registry tracking which nodes are serving which layer ranges for which models. A greedy cover algorithm identifies complete pipelines — sets of nodes whose layer ranges collectively cover all layers of the target model with no gaps. Token issuance for inference contributions is calculated based on the verified FLOPs delivered by each node's layer slice, using the same hardware-normalized formula defined in Section 4. This makes the token economy architecturally honest: earned tokens are backed by compute that directly contributes to real AI inference.

3.4.2 Centralized Fallback Layer

The centralized fallback layer provides inference capacity when the volunteer pipeline lacks sufficient online nodes to complete a request. It is backed by a self-hosted model deployment or a commercial inference API, and operates at a higher token cost per request, reflecting the absence of volunteer subsidy. The fallback layer ensures that token holders always have a path to redeem their credits regardless of volunteer network availability, preserving the access-equity guarantee of the framework.

3.4.3 Hybrid Routing

The coordination layer implements a routing policy that selects between the two backends based on current network conditions:

$$\text{backend} = \begin{cases} \text{Distributed} & \text{if } N_{\text{nodes}} \geq N_{\min} \text{ and coverage} = \text{complete} \\ \text{Fallback} & \text{otherwise} \end{cases} \tag{1}$$

where N_{nodes} is the number of online nodes collectively covering all required model layers, $N_{\min}$ is the minimum node count for a complete serving chain, and coverage completeness is verified by the shard scheduler before dispatching. Token costs differ between backends, with distributed pipeline requests costing fewer tokens to incentivize volunteer network growth.

This hybrid architecture allows the framework to provide reliable inference access from day one while progressively shifting load to the volunteer network as it grows, aligning the system's operational carbon footprint with its access-equity objectives.

3.5 System Interaction Flow

A complete interaction cycle proceeds as follows. A contributor installs the client, completes benchmarking, and registers with the coordination layer. The coordination layer assigns a hardware multiplier and records the node in the shard registry with its assigned layer range and activation server endpoint. The client launches a local activation server that accepts forward pass requests from the pipeline executor.

When an inference request arrives, the coordination layer's shard scheduler queries the registry to identify a complete pipeline — a set of nodes whose layer ranges collectively cover the target model. If a complete pipeline exists, the pipeline executor tokenizes the input at the first node, passes activations sequentially through each node's layer slice, and collects the final output from the last node. FLOPs delivered by each node are measured and reported; the coordination layer issues tokens to each contributing node proportional to their verified FLOPs, hardware multiplier, and reliability factor. Tokens are simultaneously deducted from the requester's balance.

If no complete pipeline exists, the request falls back to the centralized inference service. The coordination layer periodically dispatches inference challenge tasks to registered nodes to verify reliability, updating each node's reliability factor based on the correctness of returned activation hashes.

4 FLOP-Based Token Formula

The token economy of the Inferix framework is anchored to a single objective unit of computational work: the floating-point operation (FLOP). By denominating both token issuance and token redemption in FLOPs, the framework establishes a transparent and auditable exchange rate between contributed compute and consumed AI inference that is independent of hardware architecture, geographic location, and market conditions.

4.1 Token Issuance

Tokens are issued to contributor nodes upon verified completion of dispatched compute tasks. The number of tokens issued for a given contribution session is defined by:

$$T_{\text{earned}} = F_{\text{delivered}} \times M_{\text{hw}} \times R_{\text{reliability}} \tag{2}$$

where:

- T_{earned} is the number of tokens credited to the contributor's wallet, expressed in gigaFLOP-tokens (GFT)
- $F_{\text{delivered}}$ is the number of floating-point operations successfully delivered and verified during the contribution session, expressed in gigaFLOPs (GFLOPs)
- M_{hw} is the hardware efficiency multiplier assigned to the contributor node, as defined in Section 5
- $R_{\text{reliability}}$ is the reliability factor of the contributor node, a scalar in the range $[0.5, 1.0]$

reflecting the node's historical task completion rate and result consistency

4.2 Token Redemption

Tokens are deducted from a contributor's wallet upon submission of an inference request. The token cost of an inference request is defined as:

$$T_{\text{spent}} = F_{\text{model}} \times N_{\text{tokens}} \times Q_{\text{precision}} \tag{3}$$

where:

- T_{spent} is the number of tokens deducted, expressed in gigaFLOP-tokens (GFT)
- F_{model} is the per-token FLOP cost of the requested AI model
- N_{tokens} is the number of output tokens generated by the inference request
- $Q_{\text{precision}}$ is a precision scaling factor: 1.0 for FP32, 0.5 for FP16, and 0.25 for INT8 quantization

4.3 Reliability Factor

The reliability factor $R_{\text{reliability}}$ adjusts token issuance based on the historical trustworthiness of a contributor node:

$$R_{\text{reliability}} = 0.5 + 0.5 \times \left(\frac{C_{\text{completed}}}{C_{\text{assigned}}} \times V_{\text{consistency}} \right) \tag{4}$$

where $C_{\text{completed}}$ is the number of tasks successfully completed, C_{assigned} is the total tasks assigned in the trailing evaluation window, and $V_{\text{consistency}}$ is a result consistency score in $[0, 1]$ measuring agreement with redundant verification nodes.

4.4 Token Supply Properties

The total token supply in circulation is strictly bounded by the total verified FLOPs contributed to the network. No tokens are issued without a corresponding verified compute contribution. Tokens are non-transferable and expire after a defined inactivity period, keeping the circulating supply closely coupled to active network participation.

5 Hardware Contribution Multiplier

The hardware contribution multiplier M_{hw} normalizes token issuance across heterogeneous contributor hardware. Raw FLOP counts alone are insufficient as a basis for fair token

issuance because floating-point throughput varies not only in magnitude across hardware classes but also in practical utility for AI workloads.

5.1 Benchmarking Methodology

Upon registration and following any hardware change event, the contributor client executes a standardized benchmarking suite consisting of three components.

The first component measures sustained mixed-precision matrix multiplication throughput using matrix dimensions representative of transformer attention and feed-forward layers. The second component measures memory bandwidth under sequential and strided access patterns representative of large embedding lookups and KV-cache operations. The third component measures batch inference latency across a range of batch sizes using a reference model of defined complexity.

The three benchmark scores are combined into a composite AI Performance Index (API):

$$\text{API} = 0.5 \times S_{\text{matmul}} + 0.3 \times S_{\text{memory}} + 0.2 \times S_{\text{latency}} \tag{5}$$

where S_{matmul}, S_{memory}, and S_{latency} are each normalized to the baseline hardware class score.

5.2 Multiplier Assignment

The hardware contribution multiplier is derived from the composite API score by mapping it to a defined hardware class tier. Table 1 presents the baseline multiplier assignments for representative hardware classes.

Table 1: Hardware Contribution Multiplier Tiers

Hardware Class	**Representative Devices**	**Multiplier**
Mobile / Edge	Smartphone SoCs, Raspberry Pi	0.05×
CPU Only	Modern server and desktop CPUs	0.10×
Entry Consumer GPU	GTX 1660, RX 580	0.50×
Mid Consumer GPU	RTX 3060, RX 6700 XT	1.00×
High Consumer GPU	RTX 4080, RX 7900 XTX	2.00×
Prosumer GPU	RTX 4090, RTX 6000 Ada	3.00×
Professional Accelerator	A40, L40S	5.00×
Data Center Accelerator	A100, H100, H200	8.00×

Multiplier assignment is not strictly discrete — nodes whose API score falls between tier boundaries receive a linearly interpolated multiplier value, preventing cliff effects at tier boundaries.

5.3 Multiplier Calibration and Drift Correction

Hardware multipliers are not static. The coordination layer performs periodic global recalibration triggered by the registration of a significant volume of nodes belonging to a previously unrepresented hardware class, or a sustained deviation between predicted and observed task completion rates across any hardware tier. Recalibration adjusts multiplier values prospectively — previously issued tokens are not retroactively modified.

5.4 Resistance to Benchmark Manipulation

The benchmarking suite is designed to resist manipulation through three mechanisms. First, benchmarks are executed under controlled conditions including fixed clock frequencies, disabled concurrent applications, and a fixed random seed. Second, the coordination layer maintains a reference distribution of API scores for each known hardware model, flagging significant deviations for supervised re-benchmarking. Third, hardware changes trigger a quarantine period governed by Equation 6, as described in Section 6.

6 Security and Fairness

The Inferix framework operates in an adversarial environment in which rational participants may attempt to maximize token earnings while minimizing genuine compute contribution. We organize the threats into four categories: compute fraud, identity attacks, token attacks, and fairness violations.

6.1 Compute Fraud

6.1.1 Result Fabrication

The most direct form of compute fraud is result fabrication, in which a malicious node returns plausible-looking results without actually executing assigned tasks. The framework mitigates this through redundant verification: a random subset of tasks, selected without advance notice, are simultaneously assigned to two or more independent nodes. Results are compared using a defined consistency metric, and nodes exhibiting persistent divergence are suspended from the contributor pool.

6.1.2 Result Interpolation

A more sophisticated attack involves executing a subset of tasks genuinely and approximating results for the remainder. The framework mitigates this by embedding cryptographic challenge tasks within assigned workloads — tasks whose correct results are known to the coordination layer in advance and whose outputs cannot be approximated without genuine execution.

6.1.3 Emulation and Virtualization Fraud

A contributor may attempt to register a high-performance hardware profile while executing tasks on lower-performance hardware. The benchmarking controls of Section 5 mitigate this by enforcing hardware-level performance signatures. Additionally, the coordination layer monitors task completion times against expected throughput for the declared hardware class.

6.2 Identity Attacks

6.2.1 Sybil Attacks

A Sybil attack involves registering multiple independent node identities to multiply token earnings. The framework mitigates this through hardware fingerprinting: during benchmarking, the client generates a fingerprint derived from device identifiers, firmware signatures, and measured hardware performance characteristics. Nodes sharing a hardware fingerprint are consolidated under a single identity.

6.2.2 Identity Theft

Wallet keys are generated locally and never transmitted in plaintext. All token deduction authorizations are signed with the contributor's private key and verified against the registered public key. Contributors are encouraged to enable hardware-backed key storage where available.

6.3 Token Attacks

6.3.1 Token Forgery

The append-only, hash-chained ledger prevents token forgery by making retrospective modification detectable through hash chain invalidation. All issuance events are signed by the coordination layer's private key.

6.3.2 Replay Attacks

Each deduction authorization includes a monotonically increasing sequence number and timestamp. The inference cluster maintains a cache of recently processed sequence numbers and rejects any resubmitted authorization.

6.4 Hardware Swap Attacks

The quarantine period imposed following hardware changes eliminates the economic incentive for hardware swap attacks. The quarantine duration is defined as:

$$Q_{\text{duration}} = Q_{\text{base}} \times \log_2\left(\frac{M_{\text{new}}}{M_{\text{old}}}\right) \tag{6}$$

where Q_{base} is the base quarantine duration, M_{new} is the multiplier of the newly registered hardware, and M_{old} is the multiplier of the previously registered hardware. This formula produces a quarantine period that scales logarithmically with the magnitude of the hardware upgrade.

6.5 Fairness Considerations

6.5.1 Hardware Inequality

The baseline multiplier of 1.0× assigned to mid-tier consumer GPUs ensures the most common contributor hardware class is treated as the norm. The inclusion of CPU-only and mobile device tiers ensures that contributors without dedicated GPU hardware can participate meaningfully.

6.5.2 Temporal Fairness

A guaranteed minimum task allocation policy ensures every active node receives a minimum task assignment rate regardless of its geographic electricity cost profile.

6.5.3 Token Expiry Fairness

The expiry policy defines a minimum token lifetime of one year from issuance, with a 90-day expiry warning issued to the contributor's client before any tokens are removed from the ledger.

7 Energy and Grid Implications

The environmental impact of AI infrastructure has emerged as a significant concern. The Inferix framework, by distributing computation across a globally dispersed volunteer network, exhibits fundamentally different energy characteristics from centralized alternatives.

7.1 The Energy Profile of Centralized AI Infrastructure

Modern AI data centers are characterized by high, sustained power draw concentrated in a small number of geographic locations. A facility housing ten thousand H100 accelerators draws approximately 70 megawatts of continuous power. Because these facilities operate continuously and cannot relocate workloads in response to grid conditions, their carbon intensity is fixed by the emissions profile of the regional grid. Furthermore, centralized facilities must provision for peak demand, maintaining idle capacity during off-peak periods and consuming standby power without delivering useful computation.

7.2 The Emergent Energy Properties of Distributed Volunteer Compute

The marginal energy cost of the framework is limited to the incremental power draw of executing assigned compute tasks on hardware that would otherwise be idle. Because contributor nodes are geographically distributed across dozens of time zones, the aggregate compute supply available at any given moment is drawn from a continuously shifting pool of nodes currently in their local off-peak period. This temporal diversity produces a passive load-following effect without requiring any explicit energy-aware coordination.

7.3 Grid-Aware Task Scheduling

The coordination layer amplifies the passive load-following effect through an active grid-aware scheduling policy. Grid state data is obtained from publicly available APIs provided by regional grid operators including ENTSO-E, the US Energy Information Administration, and equivalent national operators. The grid-aware scheduling priority score for a candidate node is defined as:

$$P_{\text{grid}} = w_1 \times (1 - L_{\text{region}}) + w_2 \times (1 - C_{\text{region}}) + w_3 \times R_{\text{region}} \tag{7}$$

where L_{region} is the current grid load factor normalized to $[0, 1]$, C_{region} is the current marginal carbon intensity normalized to $[0, 1]$, R_{region} is the current renewable generation fraction in $[0, 1]$, and $w_1 = 0.4$, $w_2 = 0.4$, $w_3 = 0.2$ are weighting coefficients. The grid priority score is treated as a tiebreaker among nodes of equivalent performance class, ensuring that grid-aware scheduling does not materially degrade compute quality or latency.

7.4 Carbon Accounting

The coordination layer maintains a carbon accounting ledger recording the estimated carbon intensity of each compute task based on grid conditions at the time of execution. This ledger enables reporting of aggregate carbon metrics to participants and external auditors. Carbon accounting estimates are derived from grid operator data and published emissions factors and are subject to the measurement uncertainties inherent in those sources.

7.5 Comparison with Centralized Alternatives

Several structural observations support the conclusion that the distributed volunteer model exhibits favorable energy characteristics. First, the marginal energy cost of utilizing already-deployed consumer hardware is lower than the full lifecycle energy cost of purpose-built data center hardware. Second, the temporal distribution of volunteer compute means that a

significant fraction of contributed computation occurs during periods of grid surplus. Third, the inference cluster can be deliberately located in a region with high renewable penetration and operated at high utilization, avoiding the idle capacity losses of demand-provisioned centralized facilities. Rigorous empirical validation requires deployment data identified as a priority for future work.

8 Ecosystem Sustainability and Monetization

A decentralized compute framework predicated on volunteer contribution must address the question of long-term viability: what sustains the infrastructure, incentivizes continued participation, and funds operational costs without compromising access-equity objectives.

8.1 The Cooperative Economics of Inferix

The Inferix framework is structurally analogous to a consumer cooperative in which participation confers both obligation and benefit. Unlike cryptocurrency-based systems whose participation rates are sensitive to token price volatility, the framework offers a contribution incentive — AI inference access — whose value is stable and directly tied to the contributor's own needs. This produces a participant base whose engagement is driven by genuine demand rather than financial speculation, resulting in more stable and predictable network capacity.

8.2 Operational Cost Structure

Inference cluster costs dominate the operational budget, scaling directly with inference demand and token redemption rates. Coordination layer infrastructure costs are comparatively modest, representing a small fraction of total operational expenditure at the scale of tens of thousands of contributor nodes. Network bandwidth costs are similarly modest, as model weights are cached locally on contributor nodes following initial download.

8.3 Revenue Mechanisms

The framework supports three revenue mechanisms ordered by their alignment with the access-equity objective.

8.3.1 Token Top-Up Purchases

Contributors and non-contributors may purchase additional tokens at a published rate reflecting the true marginal cost of inference cluster operation. The pricing structure incentivizes contribution over purchase by setting the effective cost of purchased tokens modestly above the implicit value of tokens earned through contribution at the baseline hardware class.

8.3.2 Enterprise Access Tier

Organizations requiring guaranteed inference capacity, service level agreements, priority queuing, or private model deployments may subscribe to an enterprise access tier. Reserved enterprise capacity is provisioned as an additive allocation above baseline capacity available to token holders, ensuring that enterprise subscriptions expand total inference capacity rather than competing with contributor access.

8.3.3 API Access for Developers

Developers building applications on the framework's inference capabilities may access the inference API directly at a per-FLOP rate. Developer API access is priced competitively with commercial inference providers but benefits from the cost advantages of the framework's volunteer-subsidized compute base.

8.4 Sustainability Flywheel

The three revenue mechanisms collectively fund a sustainability flywheel in which revenue growth enables infrastructure expansion, which increases inference capacity, which attracts more contributors and users, which generates more revenue. This flywheel is self-reinforcing under the condition that the marginal cost of inference capacity remains below the marginal revenue generated by each additional unit of inference demand — a condition that the volunteer-subsidized compute base makes structurally favorable relative to fully commercial alternatives.

8.5 Governance and Long-Term Stewardship

The long-term sustainability of the framework depends on governance structures that preserve access-equity objectives as the ecosystem scales. We propose that the framework be governed by a public charter codifying three inviolable commitments: that tokens earned through compute contribution will always be redeemable for inference at the defined exchange rate; that the token system will never be converted to a financially tradeable instrument; and that the open-source codebase will remain freely available and forkable under a permissive license. Because the framework is open source, any violation of the charter can be remedied by the community deploying an independent instance — this fork threat serves as a structural check on operator behavior analogous to exit rights in cooperative governance.

9 Prototype Implementation and Empirical Observations

The Inferix framework has been implemented as an open-source prototype and tested in a small-scale deployment to validate the core architectural claims. This section reports observations from that deployment.

9.1 Implementation

The coordination layer is implemented as a FastAPI application backed by PostgreSQL for persistent storage of node registrations, ledger entries, and task records. The contributor client is distributed as a single cross-platform Python script requiring only the `requests`, `fastapi`, and `uvicorn` libraries, with optional `torch` and `transformers` dependencies for production inference serving. The shard registry, pipeline scheduler, and pipeline executor are implemented as in-process modules within the coordination layer. The deployment is containerized using Docker Compose and has been validated on a self-hosted NAS server.

9.2 Distributed Pipeline Validation

A two-node pipeline serving LLaMA 3 8B was established, with Node A (layers 0–15) and Node B (layers 16–31) operating on separate physical machines on a local area network. The coordination layer's shard scheduler correctly identified the complete pipeline and routed inference requests through it. End-to-end inference latency across the two-node pipeline was measured at approximately 486ms for a 20-token generation, which is consistent with the expected overhead of HTTP-based activation passing between nodes on a local network.

9.3 Token Economy Validation

The token ledger was validated under continuous operation across multiple sessions. The hash-chained ledger maintained integrity across 30+ consecutive entries. Token issuance to contributing nodes following distributed inference requests was verified, with FLOPs-proportional token distribution confirmed across both pipeline nodes. The ledger's `chain_valid` field remained `true` throughout testing, confirming tamper-evidence of the append-only structure.

9.4 Reliability Challenge System

The inference challenge mechanism was validated by dispatching known-prompt challenges to registered nodes and verifying that correct activation hashes passed verification while incorrect hashes reduced the node's reliability factor. The suspension threshold of 0.52 with a minimum of 10 completed tasks before suspension activation was validated to prevent premature suspension from isolated failures.

9.5 Auto-Reconnect Behaviour

Following coordination layer restart, registered shard nodes detected the absence of their registration via the heartbeat endpoint's 404 response within one heartbeat interval (30 seconds) and automatically re-registered without manual intervention. This validates the resilience mechanism described in Section 3.

9.6 Observations and Limitations

The prototype validates the core token economy and distributed inference pipeline at small scale. Several limitations of the current implementation are noted. First, inference is currently served in simulation mode — nodes return synthetic activations rather than executing real transformer layer computations, as the latter requires HuggingFace model weight downloads and GPU memory allocation that were outside the scope of this initial validation. Second, the fallback inference service currently returns a stub response; integration with a production inference API is identified as immediate future work. Third, the two-node test deployment is insufficient to characterise pipeline latency at scale or to measure the energy efficiency properties described in Section 7, which require a geographically distributed node population.

10 Conclusion and Future Work

10.1 Conclusion

This paper has presented Inferix, a decentralized AI compute-sharing framework that replaces cryptocurrency-based incentive structures with a closed-loop access credit economy denominated in floating-point operations. The central contribution is the reconceptualization of decentralized compute participation not as a financial investment but as a cooperative exchange: contributors donate idle computational resources and receive in return the right to consume AI inference proportional to their genuine contribution.

The framework addresses four interconnected problems in the current AI infrastructure landscape. First, it democratizes access to AI inference by gating consumption on contribution rather than capital. Second, it eliminates the speculative volatility inherent in cryptocurrency-based systems by ensuring tokens carry no financial value. Third, it contributes to sustainable AI infrastructure by distributing computation across time zones in a manner that naturally aligns workloads with periods of low grid demand, amplified by active grid-aware scheduling. Fourth, it provides a viable path to financial sustainability through a tiered monetization model that generates revenue without compromising access-equity objectives.

The FLOP-based token formula, hardware contribution multiplier methodology, reliability

factor mechanism, and security framework collectively form a technically rigorous foundation for a system that is simultaneously fair, resistant to gaming, energy-aware, and financially sustainable.

10.2 Limitations

The prototype implementation described in Section 9 validates the core architecture at small scale but leaves several limitations unaddressed. The distributed inference pipeline currently operates in simulation mode — nodes return synthetic activations rather than executing real transformer computations — and empirical characterisation of production inference throughput, latency, and accuracy across diverse hardware configurations remains as future work. The hardware contribution multiplier table is derived from theoretical performance characteristics rather than empirical benchmarking across a representative sample of volunteer hardware. The carbon accounting model described in Section 7 depends on the availability and accuracy of regional grid operator data, which varies significantly across geographies. The sustainability flywheel analysis is based on structural reasoning rather than empirical market data, and the token economics under adversarial conditions at scale have not been formally evaluated.

10.3 Future Work

10.3.1 Empirical Validation of Token Economics

A live deployment of the framework, currently underway alongside the open-source release of this codebase, is generating empirical data on contribution patterns, token accumulation rates, inference demand, and hardware distribution across the contributor base. This data will enable rigorous validation and refinement of the token formula parameters, hardware multiplier assignments, and reliability factor dynamics described in Sections 4 and 5.

10.3.2 Distributed Pipeline Scaling

The hybrid inference architecture described in Section 3.4 currently routes requests to the volunteer pipeline when sufficient nodes are available, with fallback to centralized inference otherwise. Future work should characterize the minimum viable volunteer network size for reliable inference serving across different model architectures, and develop incentive mechanisms that accelerate volunteer node recruitment for the inference layer specifically.

10.3.3 Real GPU Kernel Integration

The current contributor client executes lightweight proxy tasks that simulate GPU computation. Future work should replace these with real GPU kernels — specifically mixed-precision matrix multiplication and transformer attention operations — so that contributed compute

directly reflects the hardware's actual AI inference capability and improves the accuracy of the hardware multiplier calibration described in Section 5.

10.3.4 Model Diversity and Community Curation

Future work should explore mechanisms for community-driven model curation in which contributors vote on hosted models using a governance token distinct from the access token.

10.3.5 Federated Coordination

Future work should explore federated coordination architectures in which multiple independent coordination nodes maintain synchronized token ledgers and task dispatch queues, eliminating the single point of failure of the current centralized model.

10.3.6 Formal Security Analysis

The security mechanisms presented are design arguments rather than formal proofs. Future work should apply formal security analysis methodologies to establish rigorous security bounds.

10.3.7 Cross-Framework Interoperability

Interoperability between Inferix and complementary frameworks such as federated learning platforms may enable hybrid architectures combining access-equity properties with privacy and training capabilities.

Acknowledgements

The author would like to thank the open-source communities behind BOINC, Bittensor, Gensyn, and the federated learning ecosystem whose work provided both technical inspiration and a landscape of prior art against which the Inferix framework is positioned.

References

[1] Borzunov, A., et al. (2022). Petals: Collaborative inference and fine-tuning of large models. *arXiv preprint arXiv:2209.01188*.

[2] OpenAI. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.

[3] Alongside, Y., & Rao, A. (2022). Bittensor. *Whitepaper*. https://bittensor.com/whitepaper

[4] Gensyn. (2022). Gensyn Technical Primer. *Whitepaper*. https://gensyn.ai

[5] Render Network Foundation. (2020). Render Network Whitepaper. https://rendernetwork.com

[6] Akash Network. (2020). Akash Network Whitepaper. https://akash.network

[7] Anderson, D. P. (2004). BOINC: A system for public-resource computing and storage. *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing*, 4–10.

[8] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 1273–1282.

[9] Bonawitz, K., et al. (2019). Towards federated learning at scale: A system design. *Proceedings of Machine Learning and Systems (MLSys)*.

[10] Ryffel, T., et al. (2018). A generic framework for privacy preserving deep learning. *arXiv preprint arXiv:1811.04017*.

[11] Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 3645–3650.

[12] Patterson, D., et al. (2021). Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*.

[13] Dodge, J., et al. (2022). Measuring the carbon intensity of AI in cloud instances. *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 1877–1894.